

SAFEPET

Revisão do projeto e plano de entrega

Sistema embarcado para demonstração de cerca virtual e leitura de aceleração.

Equipe

Marcos Vinícius
Jailton Alves
Guilherme Monteiro
Gabriel Cordeiro

Conclusão da revisão

O esboço é uma base aproveitável, mas ainda não atende integralmente ao pedido do professor. O circuito original possui ESP32, display OLED e buzzer, porém não possui sensor. O código simula fuga sem calcular uma cerca virtual e não permite retorno ao estado normal.

Este pacote entrega uma versão revisada com sensor MPU6050, cálculo de distância, retorno com histerese e comandos validados. A localização permanece simulada. A equipe precisa executar os testes integrados e validar o escopo com o professor antes de apresentar.

Como usar os materiais

1. Ler a matriz de requisitos e confirmar o problema com o professor.
2. Carregar os arquivos revisados no Wokwi e executar o roteiro.
3. Ajustar o nome da empresa e a divisão da equipe.
4. Publicar os arquivos no Git e ensaiar a apresentação.

Base analisada: <https://wokwi.com/projects/476617832747866113>

Empresa sugerida: SafePet Tecnologia. Nome, funções e tempo de apresentação são propostas, não decisões já aprovadas.

1. Requisitos do professor

Enunciado recebido: usar um sistema microcontrolador, sensores, atuadores quando necessários e displays, demonstrar criatividade, documentar todo o código e documentar empresa, equipe e produto no Git. A equipe deve discutir a dor do cliente com o professor antes de fechar o problema.

Exigência	Original	Revisão / ação
Microcontrolador	Atende: ESP32	Mantido.
Sensor	Não atende: nenhum componente sensor	MPU6050 adicionado para aceleração. Teste integrado pendente.
Display	OLED SSD1306 presente	Mantido com distância, estado e leitura do sensor.
Atuador	Buzzer presente	Mantido como alerta local.
Código documentado	Comentários parciais	Funções e limitações comentadas, README e roteiro.
Dor do cliente / criatividade	Conceito de coleira SafePet	Definir tutor como cliente e validar a dor com o professor.
Empresa, equipe e produto no Git	Não verificado: nenhum repositório fornecido	Documentação pronta. Publicação remota pendente.

Pergunta central para o professor

“Podemos apresentar uma coleira com cerca virtual por coordenadas simuladas e sensor MPU6050 para leitura de aceleração nesta etapa, deixando GPS e comunicação remota como evolução?”

Se a exigência for localização medida por sensor, o MPU6050 não resolve essa parte. Nesse caso, integrar um receptor GNSS e atualizar o projeto antes da entrega.

2. O que foi encontrado no original

Achado	Consequência	Correção entregue
Serial.available() seguido de readString()	Qualquer entrada gera fuga. O teste com x confirmou esse comportamento.	Parser de comandos e rejeição de entradas inválidas.
petFugiu vira true e não volta	Só reiniciar restaura o estado inicial.	Retorno automático quando posição simulada chega a até 80 m.
Coordenadas mudam em 0,05 grau por evento	Salto artificial de cerca de 7,8 km na região da base, sem cálculo de cerca.	Haversine e comparação com limites em metros.
Sem sensor no diagrama	Requisito de sensores não demonstrado.	Inclusão do MPU6050 e leitura de aceleração.
delay() e readString()	Loop espera durante o alarme e a leitura serial.	Temporização por millis() e buffer limitado.
Inicialização presume SEGURO	Não há evidência de localização.	Início e expiração exibem SEM POSICAO.

Montagem e comentários

As ligações originais conferem com o código: OLED SDA no GPIO21, SCL no GPIO22, endereço 0x3C, alimentação em 3V3 e GND. Buzzer: terminal 1 no GND e 2 no GPIO33. Nenhuma inversão foi identificada nessa montagem.

O comentário original que diz que a sirene “corrige o problema do hardware ESP32” não tem evidência no projeto. Na revisão, o comentário descreve apenas o comportamento implementado.

Evidência e limite

O original iniciou no simulador e registrou fuga ao receber x. A revisão passou nos testes C++ da lógica compartilhada. A importação da revisão no navegador falhou. Compilação ESP32, OLED, sensor e áudio da revisão ainda precisam de validação integrada.

3. Funcionamento da versão revisada

Localização simulada e cerca virtual

O terminal fornece latitude e longitude. A função de Haversine calcula a distância até a base (-8.0880, -34.8775). Acima de 100 m, o estado passa a FORA. Até 80 m, volta a DENTRO. Entre esses limites, conserva o estado anterior. Esses valores são parâmetros didáticos, não limites homologados para um produto.

A revisão começa SEM POSICAO. Após 30 segundos de tempo simulado sem atualização, invalida a entrada, mostra SEM POSICAO e desliga o alarme de cerca. Perder a posição não significa que o animal voltou. O último estado da cerca fica preservado internamente para a próxima leitura.

Sensor complementar

O MPU6050 compartilha o barramento I2C com o OLED, no endereço 0x68. A leitura mostra o desvio da magnitude da aceleração em relação a 1 g, em m/s². Trata-se de uma medida didática, sem diagnóstico de saúde ou classificação validada de atividade. Não determina coordenadas geográficas.

Componente / conexão	Destino no ESP32
OLED SDA e MPU6050 SDA	GPIO21
OLED SCL e MPU6050 SCL	GPIO22
OLED VCC e MPU6050 VCC	3V3 no circuito simulado
OLED GND, MPU6050 GND e ADO	GND
Buzzer terminal 1 / terminal 2	GND / GPIO33

Arquivos e bibliotecas

Copiar sketch.ino, diagram.json, libraries.txt e geofence.h de firmware/ para a cópia do Wokwi. Bibliotecas: Adafruit GFX Library, Adafruit SSD1306, Adafruit MPU6050, Adafruit Unified Sensor e Adafruit BusIO.

No hardware físico, confirmar tensão e especificações dos módulos adquiridos. A placa de desenvolvimento e o OLED representam uma bancada de demonstração, sem projeto mecânico de coleira concluído.

4. Demonstração e testes

Ação	Resultado esperado
Iniciar a revisão	SEM POSICAO, sem alarme.
Enviar c e Enter	DENTRO, distância 0 m.
Enviar f e Enter	FORA, distância ~222,4 m, bipe intermitente.
Enviar c novamente	Retorno a DENTRO e silêncio.
Enviar x ou p 91 0	Mensagem de erro, posição preservada.
c e depois p -8.0872 -34.8775	~89 m: continua DENTRO.
f e depois p -8.0872 -34.8775	~89 m: continua FORA (histerese).
MPU com X=1g, Y=0g, Z=1g	Delta a aproximadamente 4,1 m/s ² .
Enviar s ou aguardar 30 s simulados	SEM POSICAO, sem alarme de cerca.

Resultados já obtidos

Testes automatizados C++ aprovados: distância zero, deslocamento de 222,4 m, limiaries de saída e retorno, histerese, coordenadas fora de faixa, valores NaN/infinity, comando incompleto e texto extra. Os testes usam o mesmo geofence.h incluído pelo firmware.

Antes de apresentar

A equipe deve executar os testes integrados acima e registrar os resultados. O arquivo docs/TESTES_E_DEMONSTRACAO.md traz casos adicionais de falta de sensor e excesso de caracteres. Todos os testes integrados da revisão estão marcados como pendentes, para evitar confundir expectativa com evidência.

A simulação observada rodava perto de 3% da velocidade normal. Use o relógio simulado para avaliar tempos. Grave uma demonstração curta após a execução bem-sucedida, como contingência para a apresentação.

5. Empresa, produto e divisão da equipe

Proposta de empresa acadêmica

SafePet Tecnologia, nome sugerido. Público proposto: tutores de cães e gatos. Problema a validar: demora para perceber que um pet saiu de uma área definida. Produto: demonstrador embarcado SafePet, com cerca virtual simulada e leitura de aceleração.

A proposta tem potencial como demonstração acadêmica, mas ainda não comprova rastreamento real, notificação ao tutor, precisão, autonomia ou viabilidade comercial. O buzzer alerta apenas nas proximidades do dispositivo.

Integrante	Divisão sugerida
Marcos Vinícius	Problema, requisitos e organização do repositório.
Jailton Alves	Circuito e demonstração no Wokwi.
Guilherme Monteiro	Firmware, cerca virtual e comentários.
Gabriel Cordeiro	Testes, resultados e organização da apresentação.

A divisão é uma sugestão para os próximos passos. Não representa autoria individual já comprovada. Todos devem entender o circuito e a lógica.

Roteiro sugerido para os slides

Marcos apresenta problema e empresa. Jailton explica o esboço e os componentes. Guilherme explica os ajustes e a cerca virtual. Gabriel conduz os testes e as limitações. Reservar uma demonstração breve. O professor não informou tempo no trecho recebido; adaptar o ensaio à orientação da disciplina.

Dicas para defender a ideia

Explicar claramente por que é um sistema embarcado: o ESP32 executa uma função específica, recebe dados e controla saídas. Mostrar o sensor funcionando. Diferenciar o alerta local da futura comunicação com o tutor. Evitar prometer que a coleira impede a fuga ou garante a segurança do animal.

6. Entrega no Git e próximos passos

O que precisa ficar no repositório

README com empresa, equipe, problema, produto e instruções. Pasta firmware/ com os quatro arquivos revisados. Pasta original/ para preservar o esboço. Pasta tests/ para a lógica automatizada. Pasta docs/ com revisão, roteiro, resultados, PDF e apresentação. O README deve ficar na raiz.

Publicação sem compartilhar credenciais

Criar no GitHub um repositório chamado safepet-sistema-embarcado e enviar o conteúdo da pasta extraída do ZIP. Preservar as subpastas. Não enviar apenas o ZIP. O arquivo docs/GIT_PASSO_A_PASSO.md inclui a alternativa por terminal. Não houve publicação remota nesta entrega.

Ordem de execução

1. Validar problema e escopo da simulação com o professor.
2. Confirmar nome da empresa e divisão da equipe.
3. Carregar a revisão em uma cópia do Wokwi.
4. Compilar, executar o roteiro e corrigir qualquer falha integrada.
5. Atualizar o registro de testes e o link do Wokwi no README.
6. Publicar o Git e conferir o acesso do professor.
7. Ensaiar os slides com demonstração.

Evoluções úteis, após fechar a entrega acadêmica

Integrar GNSS real, tratar qualidade e perda do sinal, definir comunicação remota com o tutor, medir consumo e dimensionar bateria. Estudar tamanho, peso, resistência à água e conforto. Avaliar o buzzer junto ao animal e privilegiar avisos ao tutor. Nenhuma dessas melhorias está implementada neste pacote.

Fontes consultadas em 07/10/2026

Projeto da equipe: <https://wokwi.com/projects/476617832747866113>

MPU6050: <https://docs.wokwi.com/parts/wokwi-mpu6050>

Buzzer: <https://docs.wokwi.com/parts/wokwi-buzzer>

OLED: <https://docs.wokwi.com/parts/board-ssd1306>

Biblioteca MPU6050: https://github.com/adafruit/Adafruit_MPU6050

Requisitos e nomes dos integrantes: mensagem fornecida pelo usuário. As coordenadas de base foram herdadas do esboço e são usadas apenas como referência didática.